HTB ACADEMY

Introduction to Bash Scripting

Fiche de révision complète — Style « Learn X in Y minutes »

1. Fondamentaux & Exécution

Bash (Bourne Again Shell) est le langage de script des OS Unix. Disponible sur Windows via WSL (depuis mai 2019). Contrairement aux langages compilés, Bash est interprété : pas de compilation nécessaire. Le script est exécuté par l'interpréteur Bash, il ne crée pas de nouveau processus propre.

Structure d'un script Bash

```
#!/bin/bash          # Shebang : indique l'interpréteur à utiliser
# Commentaire        # Tout ce qui suit # est ignoré
# Les grandes composantes d'un script :
# Input/Output | Arguments/Variables/Arrays | Conditions
# Arithmétique | Boucles | Opérateurs de comparaison | Fonctions
```

Méthodes d'exécution

```
bash script.sh [args]     # Via interpréteur bash explicitement
sh script.sh [args]       # Via interpréteur sh
chmod +x script.sh        # Donner les droits d'exécution
./script.sh [args]        # Exécution directe (nécessite chmod +x)
```

Toujours vérifier les droits d'exécution avant ./script.sh

2. Arguments & Variables Spéciales

Bash permet de passer jusqu'à 9 arguments ($1-$9) directement au script. $0 est réservé au nom du script. Ces variables spéciales sont des placeholders automatiques.

Variables spéciales réservées

$0	Nom du script lui-même
$1 … $9	Arguments positionnels (max 9)
$#	Nombre total d'arguments passés
$?	Code de retour de la dernière commande
$@	Tous les arguments sous forme de liste
$$	PID du processus en cours
$!	PID du dernier processus lancé en arrière-plan

Exemple d'usage des arguments

```
#!/bin/bash
# Appel : ./script.sh ARG1 ARG2 ARG3
#                     $1   $2   $3
if [ $# -eq 0 ]
then
    echo -e "Usage: $0 <domain>\n"
    exit 1
else
    domain=$1    # Assigner l'argument 1 à une variable
fi
```

```
echo "Domaine cible : $domain"
```

■ 3. Variables & Arrays

◆ Variables

```
ma_variable='valeur'          # Déclaration (pas d'espaces autour de =)
ma_variable="valeur avec espaces"
echo $ma_variable             # Afficher
echo ${ma_variable}           # Syntaxe explicite recommandée
echo ${#ma_variable}          # Longueur (nombre de caractères)

htb='HackTheBox'
echo ${#htb}                  # → 10
```

◆ Arrays (Tableaux)

```
# Déclaration
domains=(www.site.com ftp.site.com vpn.site.com)

# Accès aux éléments
echo ${domains[0]}            # Premier élément → www.site.com
echo ${domains[1]}            # Deuxième élément
echo ${domains[@]}            # Tous les éléments
echo ${#domains[@]}           # Nombre d'éléments

# Modifier un élément
domains[0]='nouveau.site.com'

# Ajouter un élément
domains+=(extra.site.com)
```

■ *Guillemets simples/doubles = valeur unique dans l'array (espaces ignorés)*

```
# PIÈGE : guillemets groupent les valeurs
domains=("www.site.com ftp.site.com" www2.site.com)
echo ${domains[0]}  # → 'www.site.com ftp.site.com' (1 seul élément !)
echo ${domains[1]}  # → www2.site.com
```

■ 4. Opérateurs de Comparaison

◆ Comparaison d'entiers

-eq	Égal à (equal)	[$a -eq $b]
-ne	Différent de (not equal)	[$a -ne $b]
-lt	Inférieur à (less than)	[$a -lt 10]
-le	Inférieur ou égal (less or equal)	[$a -le 10]
-gt	Supérieur à (greater than)	[$a -gt 10]
-ge	Supérieur ou égal (greater or equal)	[$a -ge 10]

◆ Comparaison de chaînes

==	Égal à
!=	Différent de
-z	Chaîne vide (zero length)
-n	Chaîne non vide (non-zero length)
< / >	Ordre alphabétique (dans [[ ]])

◆ Comparaison de fichiers

`-e fichier`	Le fichier existe
`-f fichier`	C'est un fichier régulier
`-d fichier`	C'est un répertoire
`-r fichier`	Droits de lecture
`-w fichier`	Droits d'écriture
`-x fichier`	Droits d'exécution
`-s fichier`	Fichier non vide (size > 0)

◆ Opérateurs logiques

`!`	NOT (négation logique)	`[ ! -e fichier ]`
`&&`	AND (et logique)	`[[ -e $f && -r $f ]]`
`||`	OR (ou logique)	`[[ -z $a || -z $b ]]`

◆ Exemple combiné (fichier)

```
#!/bin/bash
# Vérifier existence + permissions de lecture
if [[ -e "$1" && -r "$1" ]]
then
    echo "Fichier lisible."
    exit 0
elif [[ ! -e "$1" ]]
then
    echo "Le fichier n'existe pas."
    exit 2
elif [[ -e "$1" && ! -r "$1" ]]
then
    echo "Pas de permission de lecture."
    exit 1
else
    echo "Erreur inconnue."
    exit 5
fi
```

■ 5. Exécution Conditionnelle (If / Elif / Else)

Les conditions permettent de contrôler le flux d'exécution selon des valeurs ou états.

◆ Syntaxe complète If / Elif / Else

```
#!/bin/bash
value=$1
if [ $value -gt "10" ]
then
    echo "Argument supérieur à 10."
elif [ $value -lt "10" ]
then
    echo "Argument inférieur à 10."
else
    echo "Argument invalide ou non numérique."
```

```
fi
# Erreur courante :
# if-elif-else.sh: line 5: [: HTB: integer expression expected
# → Bash ne peut pas comparer une string à un entier avec -gt/-lt
```

◆ Vérification du nombre d'arguments

```
#!/bin/bash
if [ $# -eq 0 ]
then
    echo -e "Spécifiez un domaine cible.\n"
    echo -e "Usage: $0 <domain>"
    exit 1
elif [ $# -eq 1 ]
then
    domain=$1
else
    echo -e "Trop d'arguments."
    exit 1
fi
```

■ 6. Arithmétique

Bash dispose de 7 opérateurs arithmétiques. Les calculs s'effectuent avec la syntaxe $(( )) ou avec la commande let. Les opérateurs ++ et -- permettent d'incrémenter/décrémenter.

◆ Opérateurs arithmétiques

+	Addition	$((10 + 5)) → 15
-	Soustraction	$((10 - 5)) → 5
*	Multiplication	$((10 * 5)) → 50
/	Division entière	$((10 / 3)) → 3
%	Modulo (reste)	$((10 % 4)) → 2
variable++	Incrémenter de 1	((var++))
variable--	Décrémenter de 1	((var--))

◆ Exemples pratiques

```
#!/bin/bash
# Calculs avec $(( ))
echo "Addition     : $((10 + 10))"   # → 20
echo "Soustraction : $((10 - 10))"   # → 0
echo "Multiplication: $((10 * 10))"  # → 100
echo "Division     : $((10 / 10))"   # → 1
echo "Modulo       : $((10 % 4))"    # → 2
# Incrément / Décrément
increase=1
((increase++))     # increase vaut maintenant 2
((increase--))     # increase vaut maintenant 1
# Longueur d'une variable
htb="HackTheBox"
echo ${#htb}       # → 10
# Usage dans les boucles (pentest - tracking hosts)
((hosts_up++))
((hosts_total++))
```

■ 7. Input & Output

◆ Lire une entrée utilisateur (read)

```
# Lire une entrée et la stocker dans une variable
read -p "Entrez votre nom : " nom
echo "Bonjour $nom"
# Lire un mot de passe (masqué)
read -sp "Mot de passe : " password
# Exemple menu interactif (CIDR.sh)
echo -e "Options disponibles :"
echo -e "\t1) Identifier la plage réseau."
echo -e "\t2) Pinger les hôtes découverts."
echo -e "\t3) Toutes les vérifications."
echo -e "\t*) Quitter.\n"
read -p "Choisissez une option : " opt
```

◆ echo & Redirection de sortie

```
echo "Texte simple"
echo -e "Avec \n retour à la ligne et \t tabulation"
# Redirection de sortie
commande > fichier.txt       # Écrire (écrase)
commande >> fichier.txt      # Ajouter (append)
commande 2> erreurs.txt      # Rediriger stderr
commande > /dev/null 2>&1    # Supprimer toute sortie
# tee : afficher ET écrire dans un fichier simultanément
whois $ip | grep 'NetRange' | tee -a CIDR.txt
# -a / --append : ne pas écraser, ajouter au fichier
# Pipe | : enchaîner les commandes
host $domain | grep 'has address' | cut -d' ' -f4 | tee hosts.txt
```

◆ Capturer la sortie d'une commande

```
# Substitution de commande : $( ) ou ` `
ip=$(host $domain | grep 'has address' | cut -d' ' -f4)
echo $ip
# Capturer dans un fichier ET variable
hosts=$(host $domain | grep 'has address' | cut -d' ' -f4 | tee discovered_hosts.txt)
```

■ 8. Boucles (For / While / Until)

Les boucles permettent d'itérer sur des listes, plages ou jusqu'à une condition. Essentielles en pentest pour scanner des hôtes, des ports, traiter des listes.

◆ For Loop

```
# Syntaxe de base
for variable in 1 2 3 4
do
    echo $variable
done
# Sur une plage
for i in {1..10}
do
    echo $i
done
```

```
# Sur des fichiers
for file in file1 file2 file3
do
    echo $file
done
# Pinger plusieurs hôtes (usage pentest)
for ip in "10.10.10.170 10.10.10.174 10.10.10.175"
do
    ping -c 1 $ip
done
# Version one-liner
for ip in 10.10.10.170 10.10.10.174; do ping -c 1 $ip; done
# Itérer sur un array
domains=(www.site.com ftp.site.com vpn.site.com)
for domain in ${domains[@]}
do
    echo "Scan : $domain"
done
```

◆ While Loop

```
# Exécute TANT QUE la condition est vraie
counter=0
while [ $counter -lt 10 ]
do
    ((counter++))
    echo "Counter: $counter"
done
# Usage pentest : pinger jusqu'au succès
stat=1
while [ $stat -eq 1 ]
do
    ping -c 2 $host > /dev/null 2>&1
    if [ $? -eq 0 ]
    then
        echo "$host is up."
        ((stat--))
        ((hosts_up++))
    else
        echo "$host is down."
        ((stat--))
    fi
done
```

◆ Until Loop

```
# Exécute JUSQU'À ce que la condition soit vraie (inverse du while)
counter=0
until [ $counter -eq 10 ]
do
    ((counter++))
    echo "Counter: $counter"
done
# → affiche Counter: 1 jusqu'à Counter: 10
```

◆ Contrôle de boucle

break	Sortir immédiatement de la boucle
continue	Passer à l'itération suivante

■ 9. Branches : Case Statement

Le case statement est l'équivalent du switch en d'autres langages. Il compare une variable à plusieurs patterns et exécute le bloc correspondant. Chaque cas se termine par ;; et le bloc par esac.

◆ Syntaxe du Case

```
case $variable in
    "pattern1") commande1 ;;
    "pattern2") commande2 ;;
    "pattern3") commande3 ;;
    "*") commande_defaut ;;  # wildcard = défaut
esac
```

◆ Exemple : menu interactif CIDR.sh

```
#!/bin/bash
# Afficher le menu
echo -e "Options disponibles :"
echo -e "\t1) Identifier la plage réseau du domaine."
echo -e "\t2) Pinger les hôtes découverts."
echo -e "\t3) Toutes les vérifications."
echo -e "\t*) Quitter.\n"
read -p "Choisissez une option : " opt
case $opt in
    "1") network_range ;;
    "2") ping_host ;;
    "3") network_range && ping_host ;;
    "*") exit 0 ;;
esac
# Option 1 → exécute network_range
# Option 2 → exécute ping_host
# Option 3 → exécute les deux fonctions
# Autre    → quitte le script
```

■ 10. Fonctions

Les fonctions permettent de regrouper des commandes réutilisables dans un bloc nommé. Elles améliorent la lisibilité et réduisent la taille du code. Une fois définie, une fonction peut être appelée plusieurs fois.

◆ Déclaration & Appel

```
#!/bin/bash
# Déclaration (2 syntaxes valides)
function ma_fonction {
    echo "Bonjour depuis la fonction"
}
ma_fonction2() {
    echo "Syntaxe alternative"
}
# Appel de la fonction
ma_fonction
ma_fonction2
```

◆ Passage de paramètres

```bash
#!/bin/bash
function print_pars {
    echo $1 $2 $3     # Paramètres internes à la fonction
}
one="Premier paramètre"
two="Deuxième paramètre"
three="Troisième paramètre"
print_pars "$one" "$two" "$three"
# → Premier paramètre Deuxième paramètre Troisième paramètre
```

◆ Valeurs de retour & $?

```bash
#!/bin/bash
function given_args {
    if [ $# -lt 1 ]
    then
        echo -e "Nombre d'arguments: $#"
        return 1    # Code retour = erreur
    else
        echo -e "Nombre d'arguments: $#"
        return 0    # Code retour = succès
    fi
}
# Appel sans argument
given_args
echo -e "Status: $?\n"     # → Status: 1
# Appel avec argument
given_args "argument"
echo -e "Status: $?\n"     # → Status: 0
# Capturer le résultat dans une variable
content=$(given_args "argument")
echo "Contenu: $content"
```

◆ Codes de retour standard

0	Succès
1	Erreur générale
2	Mauvais usage des builtins shell
126	Commande non exécutable
127	Commande introuvable
128	Argument invalide pour exit
128+n	Erreur fatale signal n
130	Script arrêté par Ctrl+C
255	Exit status hors limites

■ 11. Debugging

Le debugging Bash est le processus d'identification et de suppression des erreurs (bugs). Il peut aussi servir à analyser comment les erreurs sont traitées pour trouver des vulnérabilités. Bash propose deux options principales : -x (xtrace) et -v (verbose).

◆ Options de debugging

bash -x script.sh	Mode trace : affiche chaque commande exécutée (préfixée par +)
bash -v script.sh	Mode verbose : affiche le code source avant exécution
bash -x -v script.sh	Mode combiné : code source + trace d'exécution

◆ Exemple : bash -x CIDR.sh

```
$ bash -x CIDR.sh
+ '[' 0 -eq 0 ']'                    # + = commande exécutée
+ echo -e 'Spécifiez un domaine.\n'
Spécifiez un domaine.
+ echo -e Usage:
Usage:
+ echo -e '\tCIDR.sh <domain>'
    CIDR.sh <domain>
+ exit 1
# Le + indique chaque commande/fonction avec ses valeurs réelles
```

◆ Exemple : bash -x -v CIDR.sh (verbose)

```
$ bash -x -v CIDR.sh
#!/bin/bash
# Check for given argument       ← CODE SOURCE affiché
if [ $# -eq 0 ]
then
    echo -e "Spécifiez un domaine.\n"
    ...
fi
+ '[' 0 -eq 0 ']'               ← PUIS trace d'exécution
+ echo -e 'Spécifiez un domaine.\n'
Spécifiez un domaine.
+ exit 1
# Verbose = code source + étapes = debugging complet
```

■ *Le debugging peut aussi détecter des vulnérabilités : manipuler la gestion des erreurs pour injecter du code (voir modules avancés HTB).*

■ 12. Cheatsheet Complète — Toutes les Syntaxes

Shebang	#!/bin/bash	Toujours en 1ère ligne
Exécuter	bash s.sh / ./s.sh	chmod +x requis pour ./
Variable	var="valeur" / $var	Pas d'espaces autour de =
Longueur var	${#var}	Nombre de caractères
Argument N	$1 … $9	$0 = nom du script
Nb arguments	$#	
Code retour	$?	0 = succès
Array déclaration	arr=(a b c)	
Array accès	${arr[0]} / ${arr[@]}	@ = tous les éléments
Array longueur	${#arr[@]}	
If simple	if [cond]; then ... fi	
If-Elif-Else	if...elif...else...fi	
Double crochets	[[cond && cond]]	Recommandé pour &&/||

Arithmétique	$((10 + 5))	+ - * / % ++ --
For loop	for x in list; do ... done	
For plage	for i in {1..10}; do ... done	
While loop	while [cond]; do ... done	Tant que vrai
Until loop	until [cond]; do ... done	Jusqu'à vrai
Break	break	Quitter la boucle
Continue	continue	Itération suivante
Case	case $v in 'x') cmd ;; esac	Double ;; obligatoire
Fonction	function nom { ... }	Ou nom() { ... }
Appel fonction	nom arg1 arg2	
Return	return 0 / return 1	0=succès, 1=erreur
Capturer sortie	var=$(commande)	Substitution de commande
Read input	read -p "Msg : " var	
Tee	cmd | tee -a fichier.txt	-a = append
Pipe	cmd1 | cmd2	Chaîner les commandes
Debug trace	bash -x script.sh	+ devant chaque commande
Debug verbose	bash -x -v script.sh	Code + trace